



Instituto Tecnológico de Aeronáutica
Divisão de Engenharia Aeronáutica e Aeroespacial

Laboratório 1

Descida de Gradiente e Introdução ao Pytorch

Deep Learning

Autores:

João Vitor Coelho Guimarães Lima

Professor:

Prof. Marcos Ricardo Omena de Albuquerque Maximo

2 de abril de 2026

1 Introdução

Este laboratório visa implementar o **algoritmo de Descida de Gradiente** utilizando as bibliotecas *NumPy* e *PyTorch*. O desempenho dos métodos será avaliado em um problema de regressão linear para a estimativa de parâmetros físicos do movimento de uma esfera. Por tratar-se de um problema com solução analítica, o Método dos Mínimos Quadrados (MMQ) seria a abordagem convencional; todavia, o cenário serve como **prova de conceito para os algoritmos iterativos**.

O PyTorch é uma biblioteca otimizada para *Deep Learning* que, além de oferecer suporte a operações em GPU, integra mecanismos de autodiferenciação (*autograd*). Esse recurso permite o cálculo automático de derivadas parciais, etapa fundamental para a otimização de modelos via gradiente descendente.

2 Metodologia

O escopo deste trabalho compreende a otimização de funções matemáticas arbitrárias por meio de métodos iterativos, assumindo a disponibilidade de amostras da função e de seus respectivos gradientes. A implementação foi concebida de forma genérica, permitindo sua aplicação em diversos contextos de otimização, além do cenário de teste específico.

No caso de estudo, **o objetivo é determinar o coeficiente de desaceleração de uma bola em um campo de futebol de robôs**. A perda de energia cinética da bola é atribuída ao fenômeno de atrito de rolamento (*rolling friction*). Assim, pode-se determinar o coeficiente de desaceleração através do seguinte algoritmo:

1. Usar câmera e visão computacional para obter posições (x,y) da bola em cada instante.
2. Calcular velocidades em x e y usando diferenças finitas centradas (exceto no primeiro e último elementos, em que deve-se usar derivadas **forward** e **backward**, respectivamente):

$$v_x[k] = \frac{x[k+1] - x[k-1]}{t[k+1] - t[k-1]} \quad (1)$$

$$v_y[k] = \frac{y[k+1] - y[k-1]}{t[k+1] - t[k-1]} \quad (2)$$

3. Calcular $v[k] = \sqrt{v_x^2[k] + v_y^2[k]}$.

4. Obter v_0 e f através de uma otimização com função de custo:

$$J([v_0, f]) = \frac{1}{2m} \sum_{k=1}^n (v_0 + ft[k] - v[k])^2 \quad (3)$$

Desse modo, tem-se os seguintes parâmetros a serem otimizados:

$$\theta_0 = v_0, \theta_1 = f \Rightarrow \theta = [v_0 \ f]^T \quad (4)$$

A distinção fundamental entre as implementações em *NumPy* e *PyTorch* reside na capacidade de autodiferenciação (*autograd*) desta última. Enquanto no *NumPy* o cálculo do gradiente da função de custo deve ser derivado analiticamente e implementado de forma manual, o ***PyTorch* automatiza a obtenção das derivadas parciais** através

de grafos de computação. Essa abstração reduz a complexidade do desenvolvimento e minimiza erros de derivação em modelos complexos.

Abaixo, detalha-se a estrutura da implementação do algoritmo de Descida de Gradiente:

2.1 Descida de Gradiente

O algoritmo de Descida de Gradiente foi empregado para determinar o vetor de parâmetros θ que minimiza a Função de Custo $J(\theta)$, conforme definido na Equação 3.

O método baseia-se na propriedade matemática de que o gradiente, $\nabla J(\theta)$, aponta para a direção de maior crescimento local da função. Portanto, para convergir ao valor mínimo, o algoritmo atualiza os parâmetros iterativamente na direção oposta ao gradiente. A regra de atualização é regida pela Equação 5:

$$\theta_{k+1} = \theta_k - \alpha \left. \frac{\partial J}{\partial \theta} \right|_{\theta=\theta_k} \quad (5)$$

Para esse caso específico, o gradiente da função custo é dado por:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \end{bmatrix} = \begin{bmatrix} \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 t_i - v_i) \\ \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 t_i - v_i) t_i \end{bmatrix} \quad (6)$$

A implementação do algoritmo de Descida de Gradiente (utilizando *NumPy*) está ilustrada no fluxograma da **Figura 1**. O processo iterativo é condicionado a dois critérios de parada principais: o número máximo de iterações (`max_iter`) e um limiar de tolerância para o custo (ϵ). A cada passo, os parâmetros θ são atualizados subtraindo-se o produto da taxa de aprendizado pelo gradiente local.

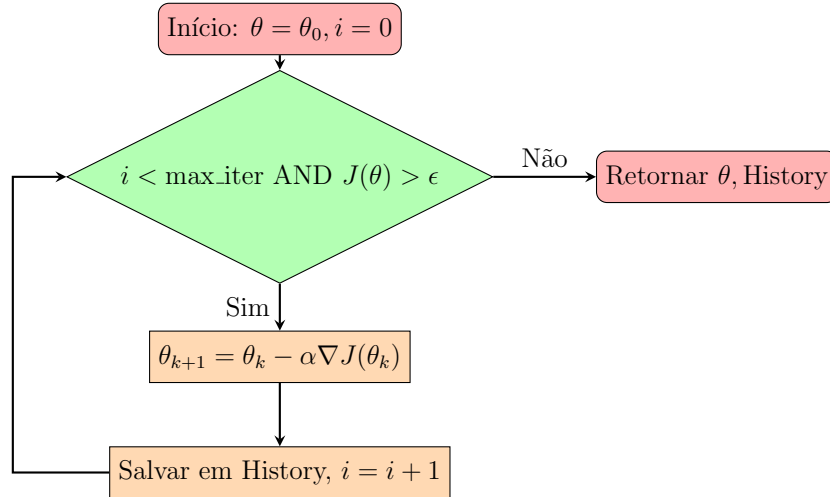


Figura 1: Fluxograma do algoritmo Gradient Descent

Após o ajuste fino dos hiperparâmetros — especificamente a taxa de aprendizado (α) e os critérios de convergência — é possível **obter o vetor de parâmetros ótimo que minimiza a função de custo** do problema proposto.

2.2 Descida de Gradiente - *Pytorch*

Para fins de comparação e exploração das funcionalidades da biblioteca *PyTorch*, implementou-se uma versão alternativa do algoritmo de otimização. A principal **vantagem** desta abordagem reside na abstração do cálculo de derivadas por meio do **motor de autodiferenciação (*Autograd*)**.

Diferente da implementação em *NumPy*, onde o gradiente analítico deve ser explicitamente fornecido, o *PyTorch* requer apenas a definição da função de custo $J(v_0, f)$, conforme a Equação 3. O algoritmo constrói um grafo computacional durante o *forward pass*, permitindo que o *backward pass* calcule automaticamente os gradientes em relação aos parâmetros de interesse (v_0 e f).

Essa implementação utiliza o otimizador nativo do *framework*, garantindo **maior eficiência computacional e escalabilidade** para modelos com maior número de parâmetros.

3 Resultados

Conforme descrito na seção anterior, a primeira implementação utilizou o algoritmo de Descida de Gradiente com operações fundamentadas puramente em *NumPy*.

Na **Figura 2**, observa-se a trajetória dos parâmetros sobre a superfície da função de custo. Nota-se que **a magnitude dos incrementos reduz-se progressivamente à medida que o algoritmo se aproxima do valor mínimo**, comportamento característico da convergência assintótica do método quando operado com uma taxa de aprendizado constante (α).

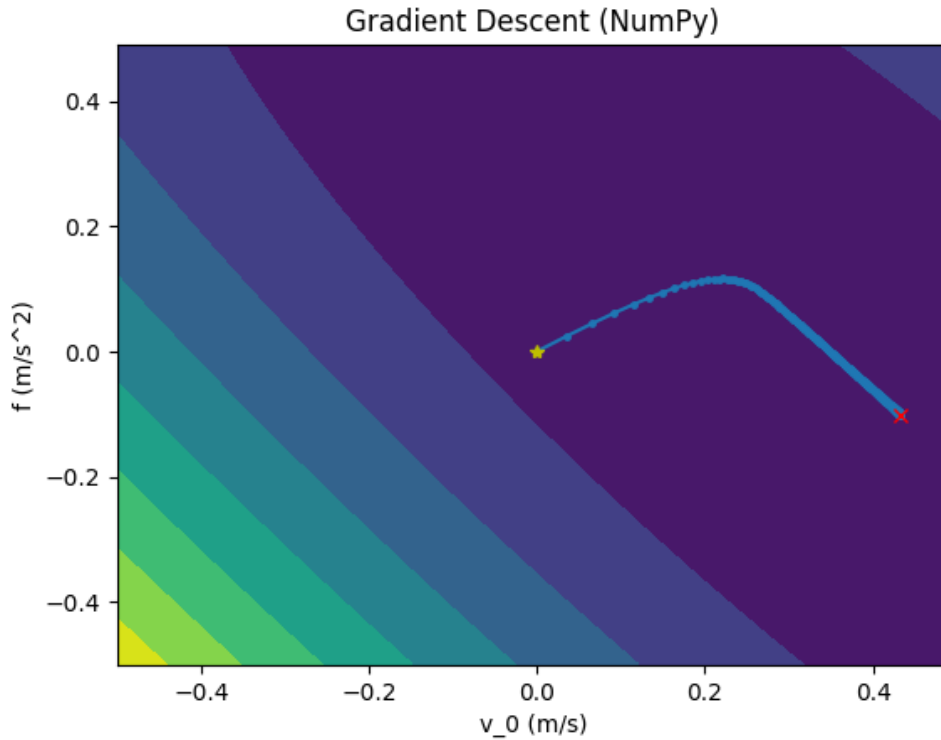


Figura 2: Descida de Gradiente usando Numpy

A análise da evolução temporal dos parâmetros, apresentada na **Figura 3**, indica uma

fase de ajuste mais acentuada nas iterações iniciais. A **estabilização** completa do sistema (convergência) ocorre **em torno da 400ª iteração**, ponto a partir do qual as variações nos valores de v_0 e f tornam-se desprezíveis, indicando que o gradiente da função de custo atingiu um valor próximo de zero.

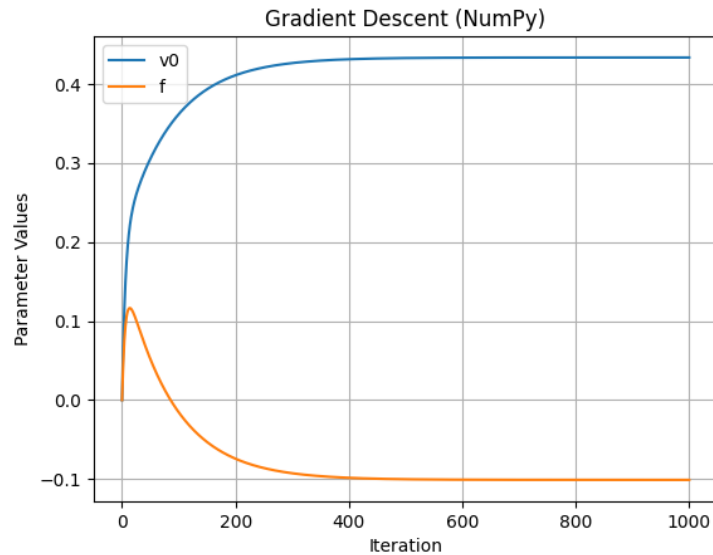


Figura 3: Evolução dos parâmetros v_0 e f ao longo das iterações via numpy

De forma análoga, a **Figura 4** apresenta a trajetória dos parâmetros sobre a superfície da função de custo utilizando o *framework PyTorch*. Observa-se que o algoritmo segue rigorosamente a direção oposta ao gradiente local. Um aspecto relevante nesta visualização é a distribuição dos pontos: a variação do espaçamento entre as iterações reflete a dinâmica de atualização do otimizador, indicando como a magnitude do passo é ajustada em função da curvatura local da superfície de custo.

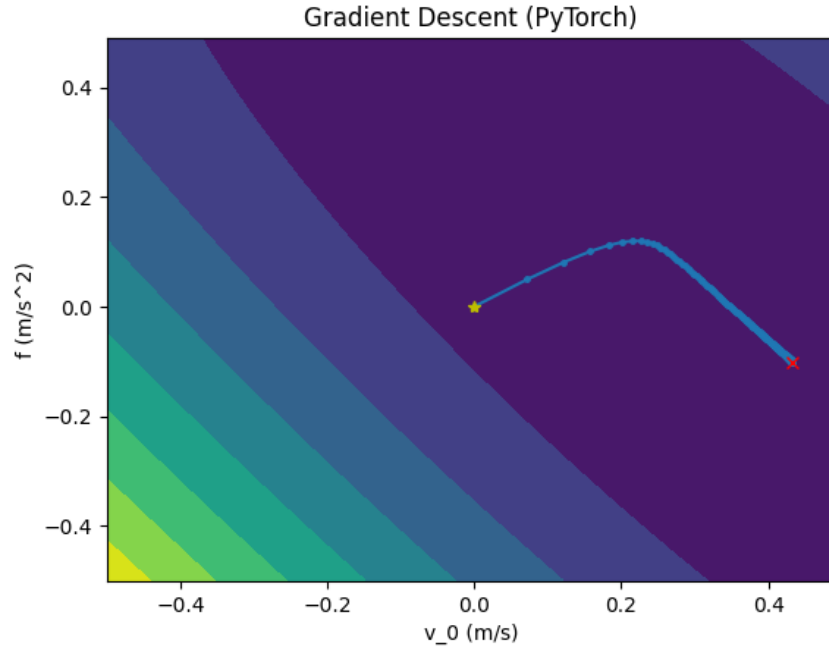


Figura 4: Trajetória da Descida de Gradiente sobre a superfície de custo utilizando PyTorch.

A análise da evolução temporal, demonstrada na Figura 5, revela um comportamento inicial similar à implementação em *NumPy*, porém com uma **convergência acelerada**, estabilizando-se em torno da 200^a iteração. Embora a lógica matemática subjacente seja idêntica, a **superioridade na velocidade de convergência** pode ser atribuída à eficiência dos algoritmos de otimização nativos do *PyTorch* e à precisão numérica do motor de autodiferenciação, que permitem uma busca mais otimizada pelo mínimo global do sistema.

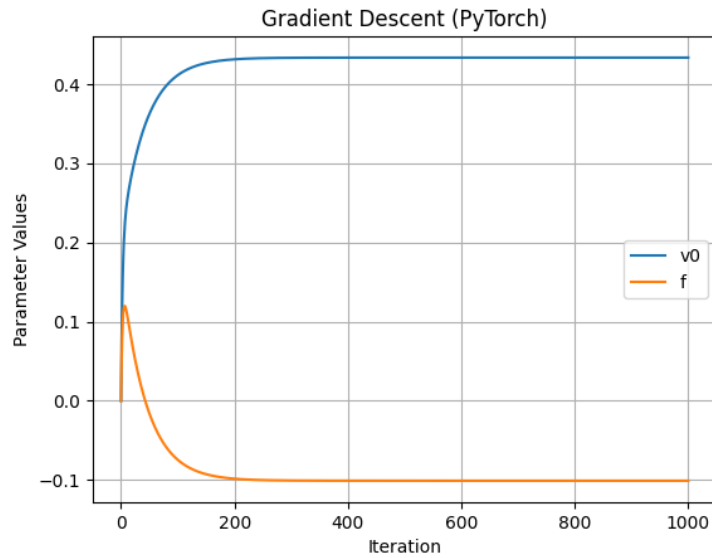


Figura 5: Evolução dos parâmetros v_0 e f via PyTorch.

A comparação direta entre as implementações em *NumPy* e *PyTorch* é apresentada na **Figura 6**. Observa-se que ambas as trajetórias são análogas, seguindo a direção oposta

ao gradiente da função de custo. Contudo, a convergência via **PyTorch** demonstra **maior eficiência**, evidenciada pelo maior espaçamento entre pontos consecutivos (passo de atualização) em comparação à implementação manual, permitindo que o sistema **atinga o ponto de equilíbrio com um menor número de iterações**.

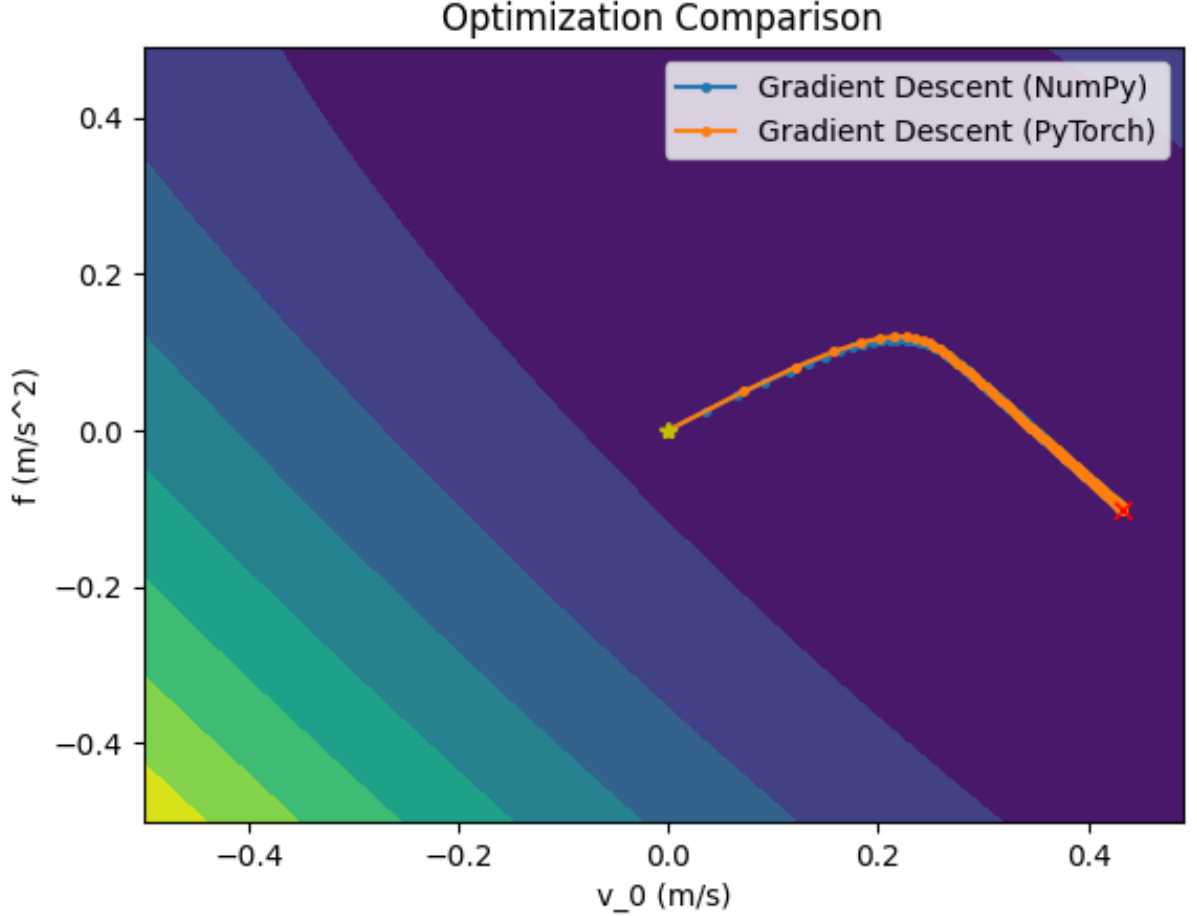


Figura 6: Comparação das trajetórias de convergência: NumPy vs. PyTorch.

Os resultados quantitativos estão compilados na **Tabela 1**. Embora os métodos converjam para valores extremamente próximos, a implementação em **PyTorch** apresentou **uma precisão numérica superior**, aproximando-se com maior fidelidade da solução analítica fornecida pelo Método dos Mínimos Quadrados (MMQ).

Método	v_0 (m/s)	f (m/s ²)
Mínimos Quadrados (Analítico)	0.433373	-0.101021
Descida de Gradiente (NumPy)	0.433371	-0.101018
Descida de Gradiente (PyTorch)	0.433372	-0.101021

Tabela 1: Comparação dos parâmetros otimizados por diferentes métodos.

Sob o aspecto prático, conforme ilustrado na Figura 7, as curvas resultantes são virtualmente sobrepostas. As discrepâncias na ordem de 10^{-6} são desprezíveis para aplicações em robótica móvel, indicando que ambos os modelos descrevem o mesmo fenômeno físico com a acurácia necessária para o controle e predição do movimento da bola.

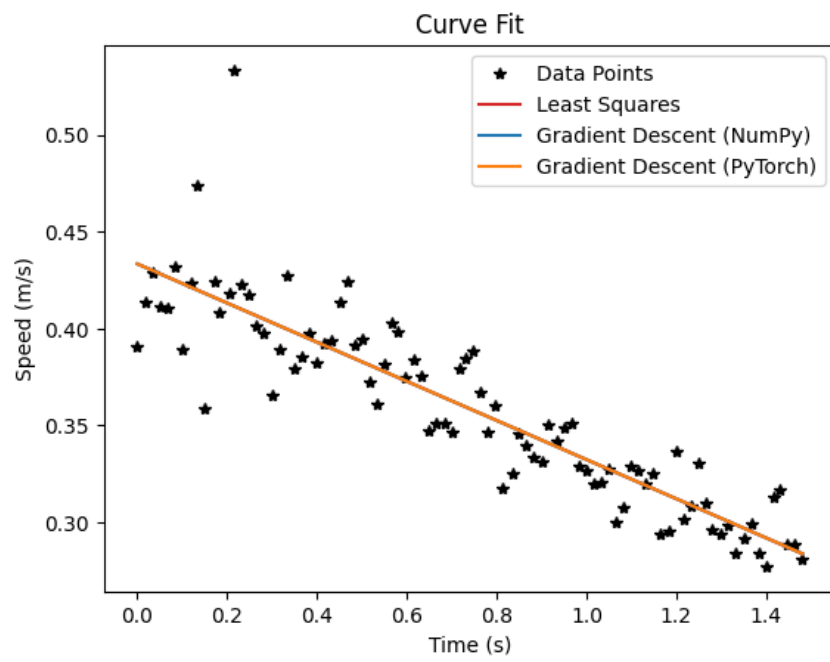


Figura 7: Ajuste do modelo linear aos dados experimentais da velocidade da bola.

4 Conclusão

Este trabalho permitiu a análise comparativa do desempenho e da implementação de algoritmos de otimização utilizando as bibliotecas *NumPy* e *PyTorch*. A implementação em *NumPy* foi fundamental para a compreensão da mecânica iterativa e do cálculo analítico dos gradientes. Todavia, os resultados demonstram que o ***framework PyTorch*** apresenta **desempenho superior** tanto em **eficiência** de convergência quanto em **precisão numérica**. Enquanto o *NumPy* demandou aproximadamente 400 iterações para estabilização, o *PyTorch* atingiu a convergência em cerca de 200 iterações, apresentando um resultado numérico mais próximo da solução analítica (MMQ).

Apesar das divergências na sexta casa decimal, **ambos os métodos mostraram-se tecnicamente adequados para a modelagem física proposta**. Para aplicações em robótica de serviços, especificamente na predição de trajetória e cálculo de forças de atrito em futebol de robôs, os erros residuais de otimização são desprezíveis frente às incertezas intrínsecas aos sensores e medições experimentais. Assim, conclui-se que a escolha entre as ferramentas deve pautar-se na complexidade do modelo e na necessidade de recursos de hardware, sendo o *PyTorch* a opção mais escalável para sistemas de alta dimensionalidade.